

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

КАФЕДРА №43

ОТЧЁТ ЗАЩИЩЁН С ОЦЕНКОЙ _____

ПРЕПОДАВАТЕЛЬ

Шумова Е.О.

должность, уч. степень, звание подпись, дата инициалы, фамилия

ОТЧЕТ ПО
ЛАБОРАТОРНОЙ РАБОТЕ №6
ТЕМА: СТАНДАРТНАЯ БИБЛИОТЕКА C++. ПОСЛЕДОВАТЕЛЬНЫЕ И
АССОЦИАТИВНЫЕ КОНТЕЙНЕРЫ. ОБОБЩЕННЫЕ АЛГОРИТМЫ
по дисциплине: «Основы программирования»

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. № 4432Кз _____ Гиппиус К.Г.
подпись, дата инициалы, фамилия

Санкт-Петербург
2026 год

Условие

Вариант 4. Реализовать шаблон класса, содержащего вектор данных, заполненный случайными числами в диапазоне $m1=0$, $m2=15$; создать второй массив как копию первого; перемешать случайным образом элементы второго массива; вычислить среднее значение всех элементов массива; перемножить поэлементно два массива; возвести в квадрат каждый элемент полученного массива. Необходимо использовать обобщенные алгоритмы, объекты-функции и предикаты, а также продемонстрировать работу не менее чем для трех типов данных.

Листинг программы

```
#include <algorithm>
#include <iomanip>
#include <iostream>
#include <numeric>
#include <random>
#include <string>
#include <type_traits>
#include <vector>

template <typename T>
struct SquareFunctor {
    T operator()(const T& value) const {
        return value * value;
    }
};

template <typename T>
struct InRangePredicate {
    T minValue;
    T maxValue;

    bool operator()(const T& value) const {
        return value >= minValue && value <= maxValue;
    }
};

template <typename T>
class ArrayProcessor {
private:
    std::vector<T> data_;
    std::vector<T> copy_;
    std::mt19937 generator_;

    T randomValue(T minValue, T maxValue) {
        if constexpr (std::is_integral_v<T>) {
            std::uniform_int_distribution<long long> distribution(
                static_cast<long long>(minValue),
                static_cast<long long>(maxValue)
            );
            return static_cast<T>(distribution(generator_));
        } else {
            std::uniform_real_distribution<double> distribution(
                static_cast<double>(minValue),
                static_cast<double>(maxValue)
            );
            return static_cast<T>(distribution(generator_));
        }
    }
};
```

```

public:
    ArrayProcessor(std::size_t size, T minValue, T maxValue) : generator_(42) {
        data_.resize(size);
        std::generate(data_.begin(), data_.end(), [this, minValue, maxValue]() {
            return randomValue(minValue, maxValue);
        });
    }

    explicit ArrayProcessor(const std::vector<T>& sourceData)
        : data_(sourceData), copy_(), generator_(42) {}

    void setData(const std::vector<T>& sourceData) {
        data_ = sourceData;
    }

    void print(const std::string& title, const std::vector<T>& values) const {
        std::cout << title << ": ";
        for (const T& value : values) {
            std::cout << std::fixed << std::setprecision(2) << value << ' ';
        }
        std::cout << '\n';
    }

    void runVariant4() {
        print("Original array", data_);

        bool allValuesInRange = std::all_of(
            data_.begin(),
            data_.end(),
            InRangePredicate<T>{static_cast<T>(0), static_cast<T>(15)}
        );
        std::cout << "All values are inside [0, 15]: " << (allValuesInRange ? "true" : "false") << '\n';

        copy_ = data_;
        print("Copied array", copy_);

        std::shuffle(copy_.begin(), copy_.end(), generator_);
        print("Shuffled copy", copy_);

        double average = std::accumulate(data_.begin(), data_.end(), 0.0) /
            static_cast<double>(data_.size());
        std::cout << "Average value: " << std::fixed << std::setprecision(2) << average << '\n';

        std::vector<T> multiplied(data_.size());
        std::transform(data_.begin(), data_.end(), copy_.begin(), multiplied.begin(),
            std::multiplies<T>());
        print("Element-wise multiplication", multiplied);

        std::transform(multiplied.begin(), multiplied.end(), multiplied.begin(), SquareFunctor<T>{});
        print("Squared result", multiplied);
        std::cout << '\n';
    }
};

int main() {
    std::cout << "Lab 6, variant 4\n\n";

    std::cout << "Type: int\n";
    ArrayProcessor<int> intProcessor(8, 0, 15);
    intProcessor.runVariant4();

    std::cout << "Type: double\n";
    ArrayProcessor<double> doubleProcessor(8, 0.0, 15.0);
    doubleProcessor.runVariant4();

    std::cout << "Type: long long\n";
    ArrayProcessor<long long> longProcessor(8, 0, 15);
    longProcessor.runVariant4();
}

```

```

std::cout << "Type: double from source data\n";
ArrayProcessor<double> sourcedProcessor({1.0, 3.5, 7.0, 9.5, 12.0, 4.0, 6.5, 10.0});
sourcedProcessor.runVariant4();

return 0;
}

```

Результаты выполнения программы

```
Lab 6, variant 4
```

```

Type: int
Original array: 5 12 15 2 11 12 9 9
All values are inside [0, 15]: true
Copied array: 5 12 15 2 11 12 9 9
Shuffled copy: 9 2 5 15 12 9 11 12
Average value: 9.38
Element-wise multiplication: 45 24 75 30 132 108 99 108
Squared result: 2025 576 5625 900 17424 11664 9801 11664

Type: double
Original array: 11.95 2.75 11.70 8.95 6.69 1.50 6.89 5.01
All values are inside [0, 15]: true
Copied array: 11.95 2.75 11.70 8.95 6.69 1.50 6.89 5.01
Shuffled copy: 11.70 8.95 11.95 1.50 5.01 6.69 2.75 6.89
Average value: 6.93
Element-wise multiplication: 139.74 24.63 139.74 13.43 33.48 10.03 18.95 34.48
Squared result: 19526.68 606.82 19526.68 180.25 1120.58 100.58 359.27 1189.04

Type: long long
Original array: 5 12 15 2 11 12 9 9
All values are inside [0, 15]: true
Copied array: 5 12 15 2 11 12 9 9

```

```

Shuffled copy: 9 2 5 15 12 9 11 12
Average value: 9.38
Element-wise multiplication: 45 24 75 30 132 108 99 108
Squared result: 2025 576 5625 900 17424 11664 9801 11664

Type: double from source data
Original array: 1.00 3.50 7.00 9.50 12.00 4.00 6.50 10.00
All values are inside [0, 15]: true
Copied array: 1.00 3.50 7.00 9.50 12.00 4.00 6.50 10.00
Shuffled copy: 3.50 6.50 10.00 1.00 4.00 12.00 9.50 7.00
Average value: 6.69
Element-wise multiplication: 3.50 22.75 70.00 9.50 48.00 48.00 61.75 70.00
Squared result: 12.25 517.56 4900.00 90.25 2304.00 2304.00 3813.06 4900.00

```

Вывод

В работе реализован шаблонный класс на основе контейнера `std::vector`. Для обработки данных использованы стандартные алгоритмы, предикат и объект-функция, а работа программы продемонстрирована для нескольких различных типов данных.